

Software Development Paradigms:

Comparing C and Python

An Introductory Overview of Procedural and
Object-Oriented Frameworks

Subject Matter: C versus Python

Prepared By: Ayash Kushwaha

Academic Track: MCA sem 1

Table of Contents

1. Understanding Programming Paradigms

- The Recipe vs. The Factory

2. Procedural Paradigm via C - A Sequential Method

- How C Works
- Code Example: Procedural C
- Why C is Great
- Where is C Used?

3. Object-Oriented Paradigm in Python - A Structured Strategy

- How Python Works using OOP
- Code Example: Object-Oriented Python
- Why Python is Great
- Where is Python Used?

4. Architectural Differences

5. Conclusion

Understanding Programming Paradigms

When we want a computer to do something, we have to give it instructions. However, just like there are different ways to cook a meal or build a house, there are different "styles" of giving instructions to a computer. In the world of computer science, we call these styles paradigms.

A programming paradigm is essentially a way of thinking about how a program should be organized. It isn't about what the computer does, but rather how the human programmer envisions the solution. If you choose the right paradigm for the job, writing the code becomes much easier and the finished product is more reliable.

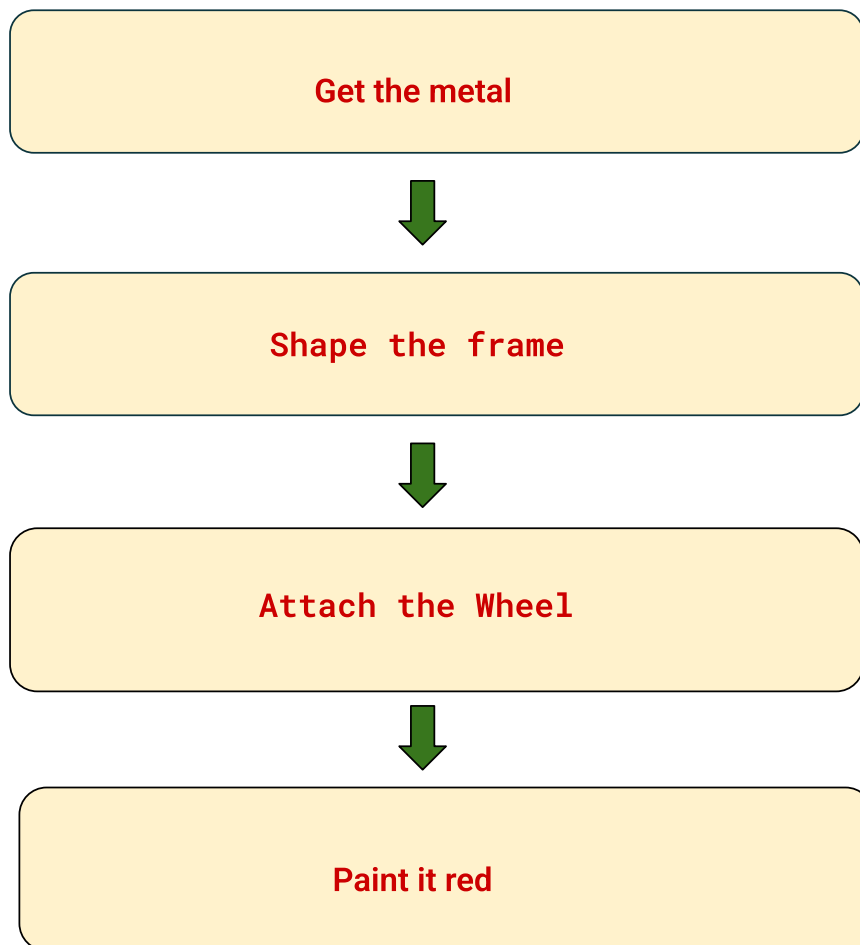
In this report, we are going to look at the two most famous ways of writing software by introducing two legendary programming languages: **C** and **Python**.

To understand the difference, let's use a simple analogy.

The Recipe vs. The Factory

Imagine you are trying to describe how to make a car.

Procedural Programming (like C) is like a detailed recipe. You list the steps one by one:



It is a step-by-step list of actions. You follow the "procedure" from top to bottom until the job is done.

Object-Oriented Programming (like Python) is like organizing a factory. Instead of focusing on the list of steps, you focus on the "things" involved. You create a "Wheel" worker, an "Engine" worker, and a "Paint" worker. Each worker knows everything about their specific part. When you want a car, these "objects" interact with each other to get the job done. The Wheel knows how to spin, and the Engine knows how to start.

Both ways get the car built, but they approach the problem from completely different angles.

Procedural Paradigm via C - A Sequential Method

The C programming language is one of the oldest and most respected languages in history. It follows the **Procedural Paradigm**. In this style, the primary focus is on the "verbs" or the actions that the computer needs to perform.

How C Works

In C, the programmer writes a series of functions (procedures). Each function is a block of code that performs a specific task. The program usually starts at the top and works its way down through these functions.

One of the key characteristics of C is that **data and actions are kept separate**. You might have a piece of data (like a list of numbers) and a separate function that sorts those numbers. The function doesn't "belong" to the numbers; it just takes them, processes them, and gives them back.

Code Example: Procedural C

Here is a basic C program demonstrating procedural style by adding two numbers inside a standalone function:

```
C/C++
#include <stdio.h>

// Procedural function to add two numbers
int addNumbers(int a, int b) {
    return a + b;
}

int main() {
    int num1 = 5;
    int num2 = 10;
    int result = addNumbers(num1, num2);
    printf("The sum is: %d\n", result);
    return 0;
}
```

Why C is Great

- **Blazing Speed:** C is incredibly fast because it is "close to the hardware." This means there is very little translation needed between the code you write and the electrical signals the computer's brain (the CPU) understands.
- **Total Control:** Using C is like driving a manual transmission car. You have total control over every gear shift. You decide exactly how the computer's memory is used.
- **Efficiency:** Because it doesn't have a lot of "extra" features running in the background, C can run on tiny devices with very little power.

Where is C Used?

C is the language used to build the foundations of the digital world.

- **Operating Systems:** The core parts of Windows, macOS, and Linux are written in C.
- **Small Devices:** The software inside your microwave, your car's engine control unit, and your digital watch is likely written in C.
- **Game Engines:** When a game needs to render millions of pixels per second, developers use C (or its cousin C++) for speed.

Object-Oriented Paradigm in Python - A Structured Strategy

Python is a modern language designed to be easy to read and powerful to use. It is most famous for supporting **Object-Oriented Programming (OOP)**. While C focuses on the actions, Python focuses on the "things" or the nouns.

How Python Works using OOP

In Python, we use something called an Object. An object is like a small container. Inside this container, we put two things together:

1. **Data:** The information (like the color of a car).
2. **Methods:** The functions or actions (like "drive" or "honk").

By keeping the data and the actions in the same package, the code becomes much more organized. If you are building a huge video game with thousands of characters, OOP allows you to treat each character as an independent "object" that knows how to take care of itself.

Code Example: Object-Oriented Python

Here is the same task in Python, written using an object-oriented approach where data and behavior are encapsulated inside a class:

Python

```
class Calculator:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def add(self):
        return self.a + self.b

# Create an instance (object) of Calculator
calc = Calculator(5, 10)
print("The sum is:", calc.add())
```

Why Python is Great

- **Readability:** Python code looks almost like English. It was designed to be easy for humans to read, not just for computers to understand.
- **The "Automatic" Life:** If C is a manual car, Python is an automatic car with self-driving features. It handles the difficult stuff, like managing memory, automatically in the background.

- **Scalability:** When projects get very large, OOP helps keep things from becoming a "spaghetti" mess of code. You can reuse objects in different parts of your program without rewriting them.

Where is Python Used?

- **Artificial Intelligence:** Almost all the modern AI and "Machine Learning" tools (like ChatGPT) are built or managed using Python.
- **Web Development:** Many of the websites you use every day, like Instagram or Pinterest, use Python on their servers.
- **Data Science:** Scientists use Python to analyze massive amounts of data to find patterns or predict the weather.

Architectural Differences

Procedural

Control : manual memory allocation

Structure : global variables accessible

Flow : top-down function calls

Object-Oriented

Control: Automatic garbage collection.

Structure: Encapsulated private state.

Flow: Message passing between objects.

S.no	Technical Phase	C {compiled language}	Python {interpreted language }
1.	Translation to machine code	Entire program is translated before running program	Code is translated line-by-line while the program is running
2.	Output File	Produces a standalone .exe or executable file	Produces .pyc bytecode, which still requires python to run
3.	Error Detection	Find syntax error during compilation (before the program starts)	Finds errors mid-execution (program will crash when it hit wrong syntax)
4.	Memory Allocation	Developer must manually allocate memory and free up RAM using malloc() and free()	Python "Garbage collector" automatically claims and frees up RAM
5.	Execution Speed	Hardware level speed maximum performance	Slower, due to line by line translation overhead

Conclusion

Both C and Python stand as vital milestones in software engineering, each serving distinct requirements based on architectural philosophy and performance needs. Procedural programming in C emphasizes sequential action, manual memory control, and direct hardware interaction, achieving unmatched execution speeds suited for operating systems, embedded systems, and game engines. Conversely, Python's object-oriented structure organizes software around encapsulated data and modular behavior, simplifying complex systems and facilitating rapid development in domains such as artificial intelligence, data science, and web applications.



Choose C for the Machine.
Choose Python for the Developer
